

Problema 2. Pinball

*Propusă de: stud. Antonie Aureliu Valentin, Facultatea de Automatică și Calculatoare,
Universitatea Națională de Știință și Tehnologie Politehnica București*

Varianta 1 de rezolvare - stud. Antonie Aureliu Valentin

Considerăm Max ca fiind valoarea maximă posibilă a lui v_i în subtaskul curent.

- $C = 1$, $1 \leq n \leq 1\,000$, $1 \leq v_i \leq 1\,000$

Pentru acest subtask, este suficient să parcurgem punctajele participanților și să calculăm, pentru fiecare, numărul de divizori, printr-o parcurgere de la 1 la v_i . Apoi, scădem din b suma divizorilor pentru a afla răspunsul la prima cerință. Complexitate $O(n \cdot Max)$.

- $C = 1$ **Fără restricții suplimentare**

Pentru abordarea de punctaj maxim, modificăm ciurul lui Eratostene și ținem minte, pentru fiecare număr non-prim, cel mai mic / mare număr prim care îl divide (SPF / LPF). Complexitatea precalculării este $O(Max \log \log(Max))$. Folosind această precalculare, putem găsi numărul de divizori al unui număr v_i în $O(\log(v_i))$. Astfel, complexitatea primei cerințe este $O(Max \log \log(Max) + n \cdot \log(Max))$.

O altă abordare pentru prima cerință este de a ne folosi de vectorul de numere prime până la $\sqrt{Max}$ în momentul în care factorizăm numerele din șir. Astfel, putem calcula numărul de divizori ai lui v_i în complexitate $O(\pi(\sqrt{v_i}))$, unde $\pi(v_i)$ = numărul de numere prime $\leq v_i$. Această abordare nu obține punctaj maxim. Complexitatea este de $O(Max \log \log Max + n \cdot \pi(\sqrt{Max}))$.

- $C = 2$, $1 \leq n \leq 50\,000$ și $1 \leq v_i \leq 100\,000$.

Pentru toate testele din acest subtask, există soluție $P \leq 100\,000$.

Putem parcurge șirul de numere și calcula numărul de divizori al fiecărui număr din șir v_i , folosind algoritmul clasic în $O(\sqrt{v_i})$. Pentru a găsi numărul de bomboane rămase, scădem din numărul total de bomboane numărul de divizori al fiecărui element al șirului. Apoi, iterăm prin toate numerele de la 1 la Max și verificăm dacă numărul curent are numărul de divizori găsit. Complexitate temporală $O((n + Max) \cdot \sqrt{Max})$.

- $C = 2$, $1 \leq v_i \leq 1\,000\,000$.

Pentru toate testele din acest subtask, există soluție $P \leq 1\,000\,000$.

Pentru acest subtask, putem modifica ciurul lui Eratostene, pentru a precalcula numărul de divizori al tuturor numerelor până la Max în $O(Max \log(Max))$, ținându-le minte într-un tablou unidimensional. Pentru a găsi numărul de bomboane rămase, scădem din bugetul b , numărul de divizori al fiecărui element al șirului. După aceea, parcurgem toate numerele de la 1 la Max și vedem dacă numărul curent are numărul de divizori găsit. Complexitate temporală $O(Max \log(Max) + n)$.

• $C = 2$ **Fără restricții suplimentare**

Pentru abordarea de 100 de puncte, plecăm de la soluția enunțată la ultimul subtask al cerinței 1 (SPF / LPF). Pe urmă, ne folosim de formula de la descompunerea în factori primi. Știm că $P = p^a q^b r^c \dots \iff d(P) = (a+1)(b+1)(c+1) \dots$. Cunoscând $d(P) = nr$, îl descompunem în factori primi, apoi, folosindu-ne de vectorul de numere prime până la $\sqrt{Max}$, obținut folosind ciurul lui Eratostene, putem genera punctajul cerut, astfel:

$$P = p^a q^b r^c \dots$$

unde $p, q, r, \dots$ reprezintă numere prime consecutive $p < q < r \dots$, iar $a+1, b+1, c+1, \dots$ reprezintă factori primi din descompunerea lui $d(P)$, $a+1 \geq b+1 \geq c+1 \dots$.

Complexitate temporală $O(Max \log \log(Max) + n \cdot \log(Max))$.

Observație: Cerința 2 poate fi de asemenea rezolvată și folosind metoda programării dinamice, aflând astfel numărul minim cu nr divizori, dar această metodă nu este în programa clasei a VI-a.